

Code Assessment of the PYUSD Smart Contracts

PUBLIC

Date [May 29, 2026](#)

Produced
for



By



CHAINSECURITY



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	6
4	Trust Model	10
5	System Considerations	13
6	Terminology	15
7	Findings	16
8	Limitations and use of report	31

1 Executive Summary

Dear M0 Team,

Thank you for trusting us to help M0 with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of PYUSDx according to [Scope](#) to support you in forming an opinion on their security risks.

M0 implements `PYUSDx`, a managed ERC-20 stablecoin featuring a configurable interest-bearing mechanism based on a continuous compounding index. The system includes extension contracts, which are ERC-20 tokens that allow users to wrap and unwrap `PYUSDx` to create white-label stablecoins. Additionally, a `SwapFacility` enables users to swap tokens between `PYUSDx`, approved extensions, and whitelisted assets. Finally, a cross-chain `Portal` facilitates token transfers across different networks using bridge adapters.

The most critical subjects covered in our audit are functional correctness, precision of arithmetic operations, and access control. Security regarding these subjects is satisfactory. We identified several edge cases that have been acknowledged and accepted as operational risks. For instance, rounding directions during transfers can reduce future yield, see [Transfers reduce unclaimed and future yield](#), and account freezing mechanisms can cause denial of service in certain yield claim paths, see [Frozen claimRecipient blocks claimFor\(\) and setAccountInfo\(\)](#), were both accepted as documented system behaviors. Among the mitigated issues, potential overflow risks in principal calculations were addressed, see [Silent Overflow of earningPrincipal in _addEarningAmount](#).

The general subjects covered are extension configuration, ecosystem integration, and behavioral consistency. Security regarding these subjects is generally good. Risks regarding untrusted MultiMint deployers in `MultiMint admin can rug-pull depositors` have also been acknowledged. Ecosystem integration remains a sensitive topic, particularly regarding the impact of yield compounding on external integrators, see [Transfer Rounding Reduces Yield for Integrators](#). We also highlight the importance of gas efficiency for future extensions to ensure reliable delivery of extension tokens, see [Future Extensions Must Limit Gas Consumption in Wrap Logic](#). Lastly, behavioral inconsistencies were identified and mitigated, see [Allowance Inconsistency Between PYUSDx and Extension Tokens for Frozen Accounts](#).

In summary, we find that the codebase provides a satisfactory level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the PYUSDx repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	27 April 2026	f56dccf2db25c0be29775e5e871d87da8c0b5996	Initial review
2	20 May 2026	876c90fc94dc02a757cf700e2d010f7e1fb465e7	Version with Fixes
3	22 May 2026	191ae5383da52f36358a807c5e13bec83dc1e809	Second round of fixes
4	28 May 2026	b33935a7c1feef9c9756e6b3e51a1efc2859b2c0	Final Version

For the Solidity smart contracts, the compiler version `0.8.34` was chosen.

```
src/
├── PYUSDx.sol
├── abstract/
│   └── RateLimiter.sol
├── core/
│   └── IssuerGateway.sol
├── platform/
│   ├── Extension.sol
│   ├── ExtensionBeacon.sol
│   ├── ExtensionBeaconProxy.sol
│   ├── ExtensionFactory.sol
│   └── projects/
│       ├── MultiMint.sol
│       └── YieldToOne.sol
├── portal/
│   ├── Portal.sol
│   ├── ReentrancyLock.sol
│   └── bridgeAdapters/
│       ├── BridgeAdapter.sol
│       └── layerZero/
│           ├── LayerZeroBridgeAdapter.sol
│           └── libraries/
│               ├── ExecutorOptions.sol
│               └── OptionsBuilder.sol
├── libraries/
│   └── PayloadEncoder.sol
└── swap/
    ├── ReentrancyLock.sol
    └── SwapFacility.sol
```

2.1.1 Excluded from scope

Any file not explicitly listed in the scope is assumed to be out of scope. In particular, third-party libraries like `openzeppelin-contracts`, `openzeppelin-contracts-upgradeable` and the M0 libraries in `common` are out of the scope of this review and are assumed to work correctly and as expected. Additionally, the LayerZero bridge is out of the scope of this review and is expected to work correctly. Lastly, the bridge configuration of the app is out of the scope of this review and is expected to follow the recommended security requirements.

3 System Overview

This system overview describes the latest version (**Version 4**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

M0 offers PYUSD, a managed ERC20 token with per-account claimable yield, two extension types that allow users to wrap and swap the tokens, as well as a bridge relying on the LayerZero infrastructure. All the deployed contracts are upgradeable.

3.1 PYUSD

The **PYUSD** token is a 6 decimals standard ERC20 token with the following additional features:

- configurable rate limited minting: addresses bearing the **ISSUER_ROLE** can mint and burn the token. The **earnerManager** can also mint token via rewards distribution (**distributeRewards()**). The **RATE_LIMIT_MANAGER_ROLE** can set a rate limit on the mint for the **earnerManager** and each of the issuers. The rate limit mechanism is a bucket that replenishes linearly with time. An unconfigured rate limit (i.e. **lastRefillTime == 0**) defaults to zero. Burns can target arbitrary non-frozen accounts.
- yield per account: a unique address, the **earnerManager** , can manage the earning rate, the fee rate and the yield recipient of each non-frozen earner. Earners track, along with their regular token balance B , their last cumulative index I_{prev} , their current earning rate r , their fee rate f , their earning principal P and the last timestamp their yield was claimed t_{prev} . Both the earner and fee rates can have a maximum value of **100%** . At the beginning of an earning session, the cumulative index starts from 1. It is continuously compounded and updated every time a transfer, yield claiming, or configuration update occurs. For any given time delta Δt with earner rate r , the current index I_{curr} is computed based on the previously checkpointed index I_{prev} as follows: $I_{curr} = I_{prev} \cdot e^{r \cdot \frac{\Delta t}{SEC_PER_YEAR}}$. The yield to be claimed Y can be computed as $Y = P \cdot I_{curr} - B = P \cdot I_{curr} - P \cdot I_{prev}$. $I_{prev} = P \cdot (I \cdot e^{r \cdot \frac{t_{curr} - t_{prev}}{SEC_PER_YEAR}}) - P \cdot I_{prev} = P \cdot I_{prev} \cdot (e^{r \cdot \frac{t_{curr} - t_{prev}}{SEC_PER_YEAR}} - 1)$ (rounded down). The fee is computed with $fee = Y \cdot \frac{f}{100}$ (rounded down). The earner's yield recipient will receive the claimable yield minus the fee, the fee is sent to the **earnerManager** . During transfers, both the balance and the earning principal are adjusted, the token amount will always be the exact transfer amount while the earning principal is computed from the transfer amount and rounding is always in favor of the protocol. Only some extensions are expected to be earners. Note that yield can be claimed permissionlessly for arbitrary earners on the **PYUSD** contract.
- freezable: freezing and unfreezing of an account is managed by the **FREEZE_MANAGER_ROLE** . While frozen, an account cannot receive nor move funds.
- pausable: pausing and unpausing of the contract is managed by the **PAUSER_ROLE** . While paused, the following are disabled: transfers, mint, burn, yield claiming, rewards distribution.
- forced transferable: forced transfers can only be triggered by the **FORCED_TRANSFER_MANAGER_ROLE** . Only a frozen account can be forced to transfer tokens.
- permit functionality (EIP-2612)
- transfer with authorization (EIP-3009)

3.1.1 Issuer Gateway

The `IssuerGateway` is expected to be one of the allowed `ISSUER_ROLE` on the `PYUSDx` contract. It implements a proposal queue for minting events, addresses bearing the `OPERATOR_ROLE` can call `proposeMint()` with an amount and a target address. The proposal then enters a queue for `mintDelay` seconds, after which it will become active, the `EXECUTOR_ROLE` can then execute the mint and create new tokens. The original proposer is checked to still bear the `PROPOSER_ROLE` before the mint can occur. The execution must happen within `mintTTL` after the proposal becomes active. Both the `mintDelay` and `mintTTL` can be updated by the `DEFAULT_ADMIN_ROLE`. These updates do not affect existing proposals as their execution window is snapshotted at creation. No minimum or maximum values are enforced for `mintDelay` and `mintTTL`, as minting in 1 transaction is not excluded. Before a proposal becomes active, the proposer can cancel it. Bearers of the `OPERATOR_ROLE` also have the power to burn tokens they hold without a time delay. Note that one `IssuerGateway` might have a rate limit configured on the `PYUSDx`, if so, the rate limit applies to the whole contract, not per `OPERATOR_ROLE`.

3.2 Extensions

Extensions are ERC20 tokens with 6 decimals, built to interact with the `SwapFacility`. They offer the `wrap()` and `unwrap()` functions that allow to lock/unlock `PYUSDx` tokens for the extension token. Similarly to the `PYUSDx` tokens, accounts can be frozen (`FREEZE_MANAGER_ROLE`) and the contract can be paused (`PAUSER_ROLE`), note however that the freeze lists are not shared and extensions are not "force transferable". Extensions are deployed behind a configurable proxy (EIP-1967 compatible), the `VERSION_MANAGER_ROLE` of each extension can pin and unpin the version of the implementation they want to use. When a version is pinned, the proxy acts as a proxy with a set logic contract address, when no version is pinned, the proxy acts as a beacon proxy. If the pinned version is set to `0`, the latest implementation will be used from the beacon for the extension type.

3.2.1 YieldToOne

The `YieldToOne` extension allows wrapping `PYUSDx` and minting the accumulated yield in the extension token to the configured yield recipient. Wrapping and unwrapping is done at a 1 : 1 rate. For the contract to accumulate yield, `YieldToOne` needs to be its own yield recipient on the `PYUSDx` contract. Only the `YIELD_RECIPIENT_MANAGER_ROLE` can trigger the minting of the excess extension token to the yield recipient. The `YIELD_RECIPIENT_MANAGER_ROLE` is also the one setting the address of yield recipient on the `YieldToOne` contract. The excess to be minted corresponds to the yield (minus fees) earned by the `PYUSDx` locked in the contract after claiming as well as potential donations:

```
excess = PYUSDx.balanceOf(YieldToOne) - YieldToOne.totalSupply()
```

While frozen, an account cannot wrap, unwrap, transfer, give allowance nor receive yield if the account is the yield recipient. While paused, wrapping, unwrapping and token transfers are not possible.

3.2.2 MultiMint

The `MultiMint` extension is built on top of `YieldToOne`, additionally, it allows users to wrap whitelisted tokens at a 1:1 rate (decimals adjusted) through the `SwapFacility`. When wrapping an asset, the effective amount (`convert_to_asset_decimals(convert_to_6_decimals(assetAmount))`) is computed and transferred from the `SwapFacility`, thus any dust from tokens with more than 6 decimals is left in the `SwapFacility`. The `SwapFacility` will then reimburse the caller with the dust in such a case. The `ASSET_CAP_MANAGER_ROLE` can configure a cap to whitelist a token, the caps represent risk exposure of the contract to the whitelisted tokens. Users can also replace the locked whitelisted tokens with `PYUSDx` through `replaceAsset()` at a 1 : 1 rate (decimals adjusted). To prevent unauthorized liquidity extraction, `replaceAsset()` is protected by an optional caller whitelist

managed by the `ASSET_CAP_MANAGER_ROLE`. Asset caps are filled on `wrapAsset()` and decreased on `replaceAsset()`.

While frozen, an account cannot wrap, unwrap, transfer, give allowance nor receive yield if the account is the yield recipient. While paused, wrapping, unwrapping, asset replacement and token transfers are not possible.

3.2.3 Extension Beacons

Each extension type (`YieldToOne` and `MultiMint`) has a dedicated beacon contract that tracks versioned implementation addresses. The `BEACON_MANAGER_ROLE` can register new versions, which immediately upgrades all extensions of that type whose proxy has no pinned version.

3.2.4 Extension Factory

Both types of extensions can be deployed permissionlessly through the factory. Users can call `deployYieldToOne()` or `deployMultiMint()`, both will deploy a beacon proxy via `CreateX` with crosschain replay enabled and a salt that depends on the deployer address and the name of the extension. The use of `CreateX` allows deployers to have the same deployment addresses across all chains for their extensions. By default, after deployment through the factory, all the extensions can be used with the `SwapFacility` as they will show as approved. The `FACTORY_MANAGER_ROLE` can whitelist extensions deployed outside of the `ExtensionFactory`, change the type of an approved extension, as well as revoke an extension.

3.3 Swap Facility

The `SwapFacility` enables users to swap tokens in the `PYUSDx` ecosystem. It allows back and forth swaps between `PYUSDx`, `YieldToOne`, `MultiMint` and allowed tokens from `MultiMint`. The `SwapFacility` is also the entrypoint for replacing assets on the `MultiMint` extensions. Notably, when wrapping assets into a `MultiMint` extension, any residual dust left over from precision conversion is refunded to the caller. The allowed extensions are the ones approved on the `ExtensionFactory`. The contract can be paused by the `PAUSER_ROLE`. While paused, all the functionalities are blocked.

3.4 Portal

The `Portal` is the entry point for cross-chain token transfers. The `sendToken()` function allows users to bridge from/to `PYUSDx` or extension tokens to other whitelisted chains. Note that the `refundAddress` should be able to handle native tokens transfers. The `Portal` abstracts the actual bridge that is used under the hood via the use of adapters (see [Bridge Adapter](#)). A default adapter for a target chain can be configured, but other adapters can be approved too. The set of adapters and their configurations is managed by the `OPERATOR_ROLE`. If the recipient of the tokens is frozen in the `PYUSDx` token on the destination, the destination tokens will be delivered to the configured `fallbackRecipient`. There is also a fallback mechanism in case the wrapping of the `PYUSDx` token into the `destinationToken` was to fail (extension does not exist, is not approved or reverts). In case of failure, the raw `PYUSDx` are delivered to the final recipient. The `Portal` is expected to bear the `ISSUER_ROLE` on `PYUSDx` and to be a trusted router on the `SwapFacility`. The contract can be paused by the `PAUSER_ROLE`. The role can pause the `send` and `receive` capabilities independently.

3.4.1 Bridge Adapter

`BridgeAdapter` is an abstract contract for cross-chain messaging backends. It defines a common interface for sending and receiving messages.

This contract can be inherited by bridge-specific adapters, which must implement the `sendMessage()` function with bridge-specific logic.

At the moment, one bridge adapter is implemented:

- `LayerZeroBridgeAdapter`: sends and receives messages through the LayerZero `EndpointV2`. The operator is set as the LayerZero delegate during `initialize()`.

4 Trust Model

Roles for each of the contracts in scope are detailed below. For every role, we have detailed the consequences of a role acting maliciously.

The `PYUSDx` contract is behind a `TransparentUpgradeableProxy` (TUP) and has the following roles. Capitalized roles may have multiple holders:

- `DEFAULT_ADMIN_ROLE` : can grant and revoke any other role besides proxy owner. **Fully trusted**.
- `ISSUER_ROLE` : expected to be attributed to the `IssuerGateway` contract and to the `Portal` contract. The role is **fully trusted**. Every holder of the role can mint the `PYUSDx` token through `mint()` up to rate limit. The role holders can `burn()` the token without limitation from any account.
- `earnerManager` role: It can attribute interest rate earnings to any account of up to ~170% yearly APY, and it can mint tokens up to the rate limit through `distributeReward()`. **Partially trusted** if rate limits are set. **Fully trusted** if no rate limit is set. Earners trust this role to set proper earning and fee rate, as well as the intended yield recipient's address. Also, the role is expected to not be frozen on `PYUSDx` because its frozen status would cause DoS for yield claims.
- `FREEZE_MANAGER_ROLE` : The freeze managers can freeze accounts. They are **partially trusted**: they can temporarily interrupt the normal operation of the protocol, but not misappropriate funds. In case of misuse, an admin can restore normal function.
- `FORCED_TRANSFER_MANAGER_ROLE` : The role grants the power to transfer frozen funds from the original holders to arbitrary recipients. It is **fully trusted**, it can misappropriate frozen funds. If it **colludes** with `FREEZE_MANAGER_ROLE`, it can misappropriate any amount of funds.
- `PAUSER_ROLE` : The role allows pausing and unpausing the token. It can interrupt the normal operation of the protocol, but not misappropriate funds. It is **partially trusted**, as misuse only temporarily interrupts normal operation, and an admin can later restore it.
- `RATE_LIMIT_MANAGER_ROLE` : allows setting rate limits for the minters. It is **partially trusted** as it can change settings but not mint or appropriate tokens. However, if it **colludes** with `ISSUER_ROLE` or `earnerManager`, they together can mint arbitrary amounts of token.
- Proxy owner: the contract is deployed behind a proxy. The proxy owner can perform arbitrary actions (minting, burning, transferring) and is **fully trusted**.

The `IssuerGateway` contract is behind a TUP and has the following roles:

- `DEFAULT_ADMIN_ROLE` : can grant and revoke any other role besides proxy owner. Can set mint delays. **Fully trusted** as it can grant privileges to mint arbitrary amounts of `PYUSDx` with no delay.
- `OPERATOR_ROLE` : can propose to mint any amount of `PYUSDx` token. **Fully trusted**, as it can misappropriate funds that should be burned.
- `EXECUTOR_ROLE` : After a delay, can approve and execute the mint amounts proposed by `OPERATOR_ROLE`. **Partially trusted** as it cannot propose mints, just approve them. If the `EXECUTOR_ROLE` and the `OPERATOR_ROLE` **collude**, they can mint arbitrary amounts (up to the rate limit configured in `PYUSDx`) if malicious proposals are not detected within the `mintDelay` time period.
- Proxy owner: the contract is deployed behind a proxy. The proxy owner is **fully trusted**.

The `ExtensionFactory` contract is behind a TUP and has the following roles:

- `FACTORY_MANAGER_ROLE` : can change the `ExtensionType` of an existing extension, including unlisting existing extensions or listing "custom" extensions with arbitrary code. When unlisting an extension, the `PYUSDx` depositors in the extension are prevented from unwrapping their tokens. **Fully trusted**. Misuse of the role can cause loss of funds if malicious extensions are approved and used by users.

- Proxy owner: the contract is deployed behind a proxy. The proxy owner can approve arbitrary potentially malicious extensions and is **fully trusted**.
- `DEFAULT_ADMIN_ROLE` : can grant and revoke any other role beside proxy owner. **Fully trusted**. Can grant the `FACTORY_MANAGER_ROLE` .

Instances of the `YieldToOne` extension have the following roles:

- `DEFAULT_ADMIN_ROLE` : Can grant and revoke any other role. **Partially trusted**. If malicious, it can disable the contract by pausing or freezing. Functionality can however be restored by upgrading the implementation contract in the `ExtensionBeacon` .
- `YIELD_RECIPIENT_MANAGER_ROLE` : can set the `yieldRecipient` address. **Partially trusted**, misconfiguration of the `yieldRecipient` can result in misappropriation of yield.
- `VERSION_MANAGER_ROLE` : Can pin a specific version of the implementation contract to be used. **Partially trusted**. If multiple versions exist and alternating between versions allows extraction of value, this role can exploit that. In case a vulnerable version is pinned, it can be overridden by upgrading the `ExtensionBeacon` .
- `FREEZE_MANAGER_ROLE` : Can freeze any account. **Partially trusted**, can interrupt normal operation of the contract but not misappropriate funds.
- `PAUSER_ROLE` : Can pause the contract. **Partially trusted**, it can interrupt normal operation, but not misappropriate funds.

Instances of the `MultiMint` extension have all the roles of `YieldToOne` , the following differences apply:

- `ASSET_CAP_MANAGER_ROLE` : can list tokens accepted to mint the extension and their caps, and manage the whitelist of callers allowed to invoke `replaceAsset()` . It is therefore **fully trusted**. If it misbehaves, it can manipulate the whitelist to DoS legitimate operators or list fake tokens to freely mint the extension, unwrap the minted tokens to misappropriate `PYUSDx` , and use the misappropriated `PYUSDx` to misappropriate other tokens in the extension. It can therefore completely drain the extension.
- `DEFAULT_ADMIN_ROLE` : is **fully trusted**, as it can grant the `ASSET_CAP_MANAGER_ROLE` .

Instances of `YieldToOne` and `MultiMint` are deployed behind `ExtensionBeaconProxy` contracts, which query the implementations from the `ExtensionBeacon` contract.

The `ExtensionBeacon` contract is behind a `TransparentUpgradeableProxy` (TUP) and has the following roles:

- `BEACON_MANAGER_ROLE` : Can modify the implementation for existing extensions which have not pinned their version. It is **fully trusted** by the unpinned extensions, as it can misappropriate the funds in any of the unpinned extensions, or freely mint extension tokens for any of the unpinned extensions. It is **not trusted** by the pinned version extensions.
- `DEFAULT_ADMIN_ROLE` : can grant and revoke other roles beside proxy owner. Like `BEACON_MANAGER_ROLE` , **fully trusted** by unpinned extensions.
- Proxy owner: the contract is deployed behind a proxy. The proxy owner can upgrade the contract and modify the implementation for any Extension token, both pinned version and unpinned version. It is therefore **fully trusted**, as it can freely mint any extension token, and misappropriate all the funds in the extension tokens.

The `SwapFacility` contract is behind a `TransparentUpgradeableProxy` (TUP), and has the following roles:

- `PAUSER_ROLE` . **Partially trusted**. Can pause the system.
- `DEFAULT_ADMIN_ROLE` , can grant and revoke roles beside proxy owner. **Partially trusted**. In practice, can indefinitely pause but it can be overridden by Proxy owner upgrading the contract.

- Proxy owner. **Fully trusted**, can upgrade contract arbitrarily, potentially drain approvals.

The `Portal` contract is behind a `TransparentUpgradeableProxy` (TUP) and has the following roles:

- `PAUSER_ROLE` : The role allows pausing and unpausing sending and receiving. **Partially trusted**. It can interrupt the normal operation of the bridge.
- `OPERATOR_ROLE` : The role can set bridge adapters. Setting malicious bridge adapters allows freely minting `PYUSDx` tokens up to the `Portal` rate limit and misappropriating bridged funds. It is **fully trusted**.
- `fallbackRecipient` : It receives tokens directed to frozen `PYUSDx` receivers. If **colluding** with `PYUSDx` `FREEZE_MANAGER_ROLE`, it can misappropriate bridged funds. It is expected to remain unfrozen in `PYUSDx`.
- `DEFAULT_ADMIN_ROLE` : Can grant and revoke any other role beside proxy owner. **fully trusted**.
- Proxy owner. **Fully trusted**. Can upgrade the contract and freely mint `PYUSDx` up to the rate limit.

The `LayerZeroBridgeAdapter` contract is behind a `TransparentUpgradeableProxy` (TUP) and has the following roles:

- `OPERATOR_ROLE` : can configure the bridge adapter, including setting an arbitrary peer. **Fully trusted**. If malicious, it can freely mint `PYUSDx` through a fake `peer` contract. It is also the delegate for the LayerZero bridge, which has the power of configuring the bridge.
- `DEFAULT_ADMIN_ROLE` : Can grant and revoke any other role beside proxy owner. **fully trusted**.
- Proxy owner. **Fully trusted**. Can upgrade the contract and freely mint `PYUSDx` up to the rate limit.

`PYUSDx` bridging depends on LayerZero correctly relaying messages. The `LayerZeroBridgeAdapter` delegate is assumed to configure LayerZero in a trust minimizing way, such that no single party (relayers, LayerZero multisig) can forge messages.

External tokens integrating with `MultiMint` are **fully trusted**. They are non-rebasing, and have no fees on transfer.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Bridging a MultiMint Extension Token May Fail Due to Insufficient PYUSDx Backing

System Consideration	Version 1
----------------------	-----------

When a MultiMint extension token is used as the source token for a cross-chain transfer, Portal calls SwapFacility.swapOut() to unwrap it into PYUSDx. MultiMint._beforeUnwrap() reverts if the contract does not hold enough PYUSDx to back the redeemed amount. If the MultiMint extension has insufficient PYUSDx liquidity at the time of bridging, the entire transaction will revert.

SC2 Cross-Chain Total Supply May Momentarily Drop

System Consideration	Version 1
----------------------	-----------

When computing the total supply of PYUSDx across all chains, observers should be aware that the aggregate value can temporarily decrease. When a cross-chain transfer is initiated, the tokens are burned on the source chain immediately, but minted on the destination chain only once the bridge message is delivered. During this in-flight window the burned amount is absent from the global supply.

SC3 Effects of Freezing SwapFacility or Portal on PYUSDx and Extensions

System Consideration	Version 1
----------------------	-----------

SwapFacility or Portal frozen on PYUSDx: If SwapFacility is frozen: swapIn(), swapOut(), swapExtensions(), and replaceAsset() all fail because each path involves a PYUSDx transfer to or from SwapFacility. swapInMultiMint() is unaffected since it only moves external assets. Portal sending PYUSDx directly still works; sending an extension token as source fails because swapOut() is called internally. Receiving with a PYUSDx destination is unaffected; receiving with an extension destination silently falls back to delivering PYUSDx due to the low-level call in _wrap(). If Portal is frozen: both sendToken() and receiveMessage() fail entirely — mint() and burn() revert on a frozen account and PYUSDx transfers to Portal are blocked. All SwapFacility and extension operations are unaffected.

SwapFacility or Portal frozen on an extension: _beforeWrap() and _beforeUnwrap() receive the original caller as account via SwapFacility.msgSender(), which resolves through the trusted-router chain (_locker). Neither SwapFacility nor Portal appears as account in those hooks. Freeze checks on extensions only catch them through _beforeTransfer(), which inspects the actual msg.sender of the ERC20 call and the recipient. If SwapFacility is frozen: swapIn() is unaffected since no extension token is transferred to or from SwapFacility. swapOut(), swapExtensions(), and replaceAsset() with an extension tokenIn all fail because _beforeTransfer() catches SwapFacility as both msg.sender and recipient of the extension transferFrom. Portal receiving to that extension is unaffected; sending with that extension as source fails. If Portal is frozen: sendToken() with that extension as source fails — _beforeTransfer() catches Portal as

`msg.sender` (the ERC20 caller) and as recipient in the initial `safeTransferFrom(user, Portal, amount)`. Receiving to that extension is unaffected since `Portal` does not appear as `account`, `msg.sender`, or recipient in the extension's transfer path. Sending `PYUSDx` directly is unaffected.

SC4 Transfer Rounding Reduces Yield for Integrators

System Consideration	Version 1
----------------------	-----------

We want to highlight the practical impact of [Transfers reduce unclaimed and future yield](#) for integrators building on top of `PYUSDx`. Every transfer slightly reduces the receiver's earning principal due to rounding, and integrators should be aware of two concrete consequences:

- **Wrap/unwrap cycles:** each `wrap()` + `unwrap()` round-trip on an extension decreases the caller's earning principal by 2 units, permanently eroding yield.
- **ERC-4626 vaults:** if `totalAssets` is computed as `balanceWithYieldOf(self)`, every deposit can marginally reduce the share price, as the principal loss from the incoming transfer offsets the deposited amount. Vaults that assume a monotonically increasing share price may be broken by this behavior.

SC5 Yield of MultiMint Can Be Reduced by Replacing PYUSDx with Asset

System Consideration	Version 1
----------------------	-----------

Even if a `MultiMint` extension has a stable TVL, its yield as a percentage of the TVL can fluctuate depending on the composition of the TVL: `PYUSDx` vs alternative collateral assets.

In particular, the yield of a `MultiMint` can be reduced by replacing `PYUSDx` with another collateral asset.

This can be performed by:

1. wrapping an asset to mint the `MultiMint` extension token.
2. unwrapping the `MultiMint` extension token to extract `PYUSDx`

Yield recipients of `MultiMint` should expect fluctuations in yield, especially at times when the alternative collateral assets trade below the peg (such that trading Asset -> `PYUSDx` through the `MultiMint` is incentivized)

SC6 Future Extensions Must Limit Gas Consumption in Wrap Logic

System Consideration	Version 1
----------------------	-----------

As described in [Low-Level Call in `\_wrap\(\)` Can Be Forced Into the Fallback Branch](#), `Portal._wrap()` forwards at most 63/64 of the available gas to `swapFacility.swapIn()`. If a future extension's wrap logic consumes a large amount of gas, an attacker could exploit this to force the low-level call to fail and silently receive plain `PYUSDx` instead of the intended extension token.

Extension developers must ensure that `swapIn()` implementations remain gas-efficient. High gas consumption directly increases the attack surface for this class of exploit.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

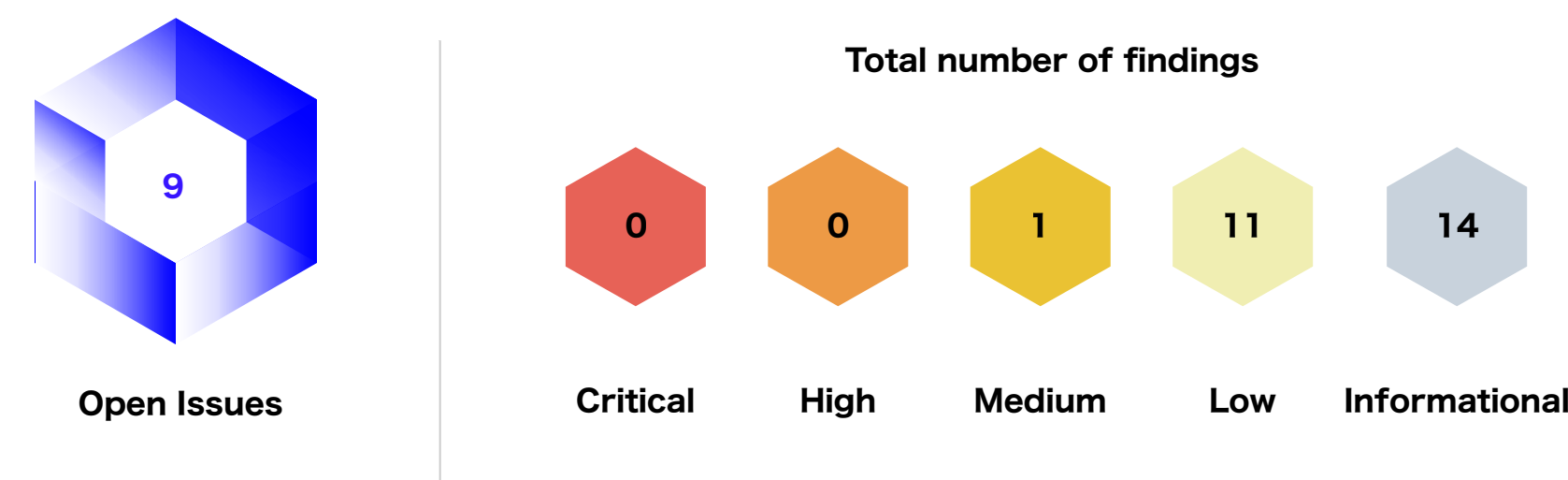
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



Info	#018	Inaccurate NatSpec	Code Corrected	✓
Info	#019	Low-Level Call in <code>_wrap()</code> Can Be Forced into the Fallback Branch	Acknowledged	⊖
Info	#020	Missing Proposer Role Validation During Mint Execution	Code Corrected	✓
Info	#021	Restricted Cancellation Logic Leads to Dangling Proposals	Code Corrected	✓
Info	#022	Silent Mapping Updates in <code>setBridgeChainId()</code>	Code Corrected	✓
Info	#023	Specification Mismatches	Specification Changed	✓
Info	#024	Extension Cannot Be Used as Collateral Asset in MultiMint	Acknowledged	⊖
Info	#025	Frozen PYUSD Account Can Still Swap Between Extensions	Acknowledged	⊖
Info	#026	Missing Consistency Check for Beacons in ExtensionFactory	Code Corrected	✓

7.2 Descriptions

#001 MultiMint Admin Can Rug-Pull Depositors

Trust	Medium	Version 1	Risk Accepted ⓘ
-------	--------	-----------	-----------------

`deployMultiMint()` is callable by anyone. Any factory-deployed extension is automatically registered as an approved extension, making it a valid swap destination in `SwapFacility`. A malicious actor could deploy a `MultiMint` and attract users `PYUSDx` deposits.

The deployer can later allow a worthless asset as collateral, and use it to drain the existing depositors (`wrapAsset()` + `unwrap()`). The deployer can perform the attack in a single transaction, so monitoring is an ineffective mitigation.

Risk Accepted

M0 is aware of this issue and stated:

Extension deployment via `ExtensionFactory` is intentionally kept permissionless. The attack surface is contained to the malicious extension itself: it does not affect existing extensions or `PYUSDx` holders who never interact with it. New deployments will be monitored.

#002 `setExtensionType()` Allows Modification of Active Extension Types

Correctness	Low	Version 1	Risk Accepted ⓘ
-------------	-----	-----------	-----------------

Version 1

The `setExtensionType()` function in `ExtensionFactory` is intended for approving or revoking extensions. However, the current logic allows an admin to change the `ExtensionType` of registered extensions (e.g., from `YIELD_TO_ONE` to `MULTI_MINT`).

If the type is modified, integrators relying on `ExtensionFactory.getExtensionType()` to determine how to interact with the extension will be misled. This discrepancy will result in unexpected execution reverts and flawed integration logic.

Version 2

The function was renamed to `registerExtension()` and a check was added to forbid direct modifications from one active type to another. However, this restriction can be bypassed by resetting the type first.

Risk Accepted

M0 is aware of this issue and stated:

We will keep it as is for flexibility.

#003 Allowance Inconsistency Between PYUSD and Extension Tokens for Frozen Accounts

Design

Low

Version 1

Code Corrected ✓

PYUSD does not check the frozen status in `approve()`, so frozen accounts can still grant allowances. Extension tokens enforce the opposite: `_beforeApprove()` reverts if either the owner or the spender is frozen. This inconsistency means a frozen account's ability to set allowances depends on which token is involved.

Code Corrected

The `_approve()` function in PYUSD has been updated to revert if either the owner or the spender is frozen.

#004 Ambiguous Semantics of `type(uint128).max` in RateLimiter

Correctness

Low

Version 1

Code Corrected ✓

In `RateLimiter.sol`, the value `type(uint128).max` is returned by `getRemainingAmount()` to indicate that an issuer is unconfigured and has unlimited capacity (up to `type(uint256).max`).

```
function getRemainingAmount(address issuer) external view returns (uint128) {
    Bucket storage bucket = _getRateLimiterStorage().issuerBuckets[issuer];

    // NOTE: Unconfigured issuers have unlimited capacity
    if (bucket.lastRefillTime == 0) return type(uint128).max;
    ...
}
```

However, the same value can be explicitly set via `setRateLimit()`, leading to a semantic overlap where the same value represents both an infinite and a finite limit. The API fails to distinguish between truly unlimited and maximum finite limit, resulting in inconsistent behaviors for the same return value.

Code Corrected

`getRemainingAmount()` now returns `0` for unconfigured issuers instead of `type(uint128).max`, removing the ambiguity between unlimited and maximum finite limit.

#005 Delegate Not Reset When Operator Role Is Revoked

Design

Low

Version 1

Code Corrected ✓

`_revokeRole()` in `LayerZeroBridgeAdapter` only resets the LayerZero delegate to `address(0)` if the current delegate matches the account being revoked. Since the operator can call `setDelegate()` to point the delegate to an arbitrary address, revoking the operator role will not reset the delegate in that case, leaving an unintended address in control of LayerZero endpoint settings. This is particularly problematic if the operator went rogue or was compromised, as revoking the role would not fully contain the damage.

Code Corrected

`_revokeRole()` now unconditionally resets the delegate to `address(0)` whenever `OPERATOR_ROLE` is revoked, regardless of the current delegate address.

#006 Frozen `claimRecipient()` Blocks `claimFor()` and `setAccountInfo()`

Design	Low	Version 1	Risk Accepted ⓘ
--------	-----	-----------	-----------------

In `PYUSDX.sol`, the internal `_claimFor()` function handles the yield distribution logic, materializing accrued yield and distributing fees. When `skipTransfer` is `false`, the function may trigger up to two `_transfer()`:

- 1. If `claimRecipient` is different from the account, the net yield is transferred to the claim recipient.
- 2. If `fee > 0`, the protocol fee is transferred to the `earnerManager`.

Because the `_transfer()` function enforces a strict `_revertIfFrozen(recipient)` check, the `_claimFor()` operation and any function (e.g. `setAccountInfo()`) that relies on it will revert if either of these destination addresses is frozen in `PYUSDX`.

This is a problem in particular when `claimRecipient` for a legitimate `account` has been frozen, as the `earnerManager` is unable to restore a valid earning state (update the `claimRecipient`, potentially change the rate) to the `account`. M0 suggests the following mitigation:

To remediate this issue, it's possible to freeze the account since we skip transfers when freezing

Then the freeze manager can then use force transfer to seize the user's balance and handle restitution of the funds and the yield + fee manually later on. It's also possible to pause the contract and then change the claim recipient since we skip transfers when the contract is paused but it's not ideal. Maybe we should also skip transfers when the claim recipient is frozen to be able to easily change the claim recipient.

This mitigation involves temporarily freezing the affected `account` before calling `setAccountInfo()`. It is problematic as it freezes an account which has no fault, potentially hampering its operation and flagging it for scrutiny, and it requires the cooperation of two other roles (`FREEZE_MANAGER_ROLE`, `FORCED_TRANSFER_MANAGER_ROLE`), which is potentially difficult to coordinate.

Risk Accepted

M0 is aware of this issue and stated:

If the `earnerManager` is frozen, the admin can call `setEarnerManager()` to resume claims. If a `claimRecipient` is frozen, yield should not be transferred to it; resolving that state requires either temporarily unfreezing the `claimRecipient` before calling `setAccountInfo()`, or freezing the account and using `forceTransfer()` to reconcile the balance sheet off-chain. Operationally, the Freeze Manager should freeze an account and its `claimRecipient` together to avoid reaching this state.

#007 Rate Limiting Defaults to Unlimited Capacity

Security

Low

Version 1

Code Corrected ✓

In `RateLimiter.sol`, the system is designed to bypass rate limiting for any unconfigured issuer. This is achieved by checking if `lastRefillTime == 0`.

```
function _enforceRateLimit(address issuer, uint256 amount) internal {
    Bucket storage bucket = _getRateLimiterStorage().issuerBuckets[issuer];

    // NOTE: Unconfigured issuers have unlimited capacity
    if (bucket.lastRefillTime == 0) return;

    // enforcement logic follows
}
```

This violates the principle of secure-by-default. If a new issuer is granted the `ISSUER_ROLE`, but the admin forgets to call `setRateLimit()`, the issuer will have unlimited minting power without any safety buffer.

Code Corrected

Unconfigured issuers now default to zero capacity.

#008 Silent Overflow of earningPrincipal in _addEarningAmount

Correctness

Low

Version 1

Code Corrected ✓

In `PYUSDx.sol`, the `_addEarningAmount()` function updates an account's `balance` and `earningPrincipal` (stored as `uint112`) within an unchecked block. While the `balance` update is safe as it is bounded by the verified `totalSupply`, the system does not enforce the 112-bit limit for individual account principals.

```
function _addEarningAmount(PYUSDxStorage storage $, address account, uint256 amount)
    internal {
    uint112 principal = _getPrincipalAmountRoundedDown(amount, _updateIndex0f(account));
    unchecked {
        $.accounts[account].balance += amount;
        $.accounts[account].earningPrincipal += principal;
    }
}
```

If the addition causes `earningPrincipal` to exceed `type(uint112).max`, it will silently wrap around. This corrupts the account's earning state, resulting in a significantly lower `earningPrincipal`. Consequently, the user will receive much less yield than they are entitled to.

Code Corrected

`_addEarningAmount()` now performs the `earningPrincipal` update using standard Solidity addition, ensuring overflow protection.

#009 Stuck Dust Due to Decimal Truncation in MultiMint

Correctness

Low

Version 1

Partially Corrected / Risk Accepted 

`MultiMint` allows users to mint extension tokens by depositing approved assets or to replace those assets with `PYUSDx`. While extension token has a fixed 6-decimal precision, the underlying assets can have different decimal counts.

The internal conversion logic in `_convertAmounts()` truncates any remainder when converting from a higher precision to a lower precision. Nonetheless, `MultiMint` still pulls the specified amounts from the user without refunding the difference. Therefore, dust accumulates across two interaction paths:

- Wrapping Assets with > 6 Decimals (e.g., DAI with 18 decimals):** The full amount is pulled from the user, but `extensionAmount` is truncated to 6 decimals, leaving the fractional remainder within the contract balance.
- Replacing Asset with < 6 Decimals (e.g., Gemini USD with 2 decimals):** In `_replaceAsset()`, a user pays a full 6-decimal `PYUSDx` amount but receives an asset amount truncated to its lower precision. For example, if a user pays 1.123456 `PYUSDx` for a 2-decimal asset, they only receive 1.12 tokens while the excess 0.003456 `PYUSDx` is still transferred to the contract.

Although the internal `assets[asset].balance` correctly tracks these full amounts (including the remainders), the balances cannot be fully cleared through normal protocol interactions because the contract is constrained by fixed-precision granularities.

Partially Corrected

`MultiMint._wrapAsset()` was updated to pull the effective amount `( _fromExtensionToAssetAmount(asset, _fromAssetToExtensionAmount(asset, amount)) )` from the `SwapFacility`. Any `asset` dust left in the `SwapFacility` will be sent back to the original caller.

Risk Accepted

No such solution was implemented for the `replaceAsset()` path. M0 is aware of this issue and stated:

The case of replacing assets with fewer than 6 decimals is risk-accepted: such assets are rare and handling them would add notable complexity to the codebase.

#010 Timelock Parameter Changes Retrospectively Modify Pending Proposal Constraints

Correctness

Low

Version 1

Code Corrected 

In `IssuerGateway.sol`, a mint proposal's execution window is not snapshotted at creation. Instead, the contract dynamically calculates the `activeAt` and `expiresAt` timestamps during `mint()` or `cancelMint()` using the current global `mintDelay` and `mintTTL` values.

Changing the global `mintDelay` or `mintTTL` alters the lifecycle of all pending proposals. A proposal that was previously expired could be reactivated if `mintTTL` increases, or a proposal that was ready for execution could be pushed back into pending if `mintDelay` increases.

Further, the `MintProposed` event emits specific `activeAt` and `expiresAt` timestamps based on the parameters at the time of proposal. Any subsequent change to the global parameters renders these emitted values incorrect.

Code Corrected

The timestamps `activeAt` and `expireAt` are now recorded within `MintProposal` struct at creation time.

#011 Transfers Reduce Unclaimed and Future Yield

Design

Low

Version 1

Risk Accepted !

The rounding behavior on transfer operations of `PYUSDx` reduces the unclaimed yield and the future yield.

The yield of an account is defined as:

```
min(principal * index / 10**12 - balance, 0)
```

When a transfer amount is received, the new `balance` and `principal` are updated as:

```
balance += amount
principal += amount * 10**12 / index //<- rounds down
```

Since the `principal` increase rounds down, after a transfer the yield decreases by 1 (corresponding to 10^{-6} `PYUSDx`), for `index` in the interval $[10^{12}, 2 \cdot 10^{12})$. In case the yield is currently 0, because

```
principal * index / 10**12 - balance <= 0
```

the principal is still rounded down, leading to a loss of future yield: it will take longer for `principal * index / 10**12` to become greater than `balance` such that yield is generated.

This can potentially be a griefing vector, and can cause issues in integrations. For example:

1. `YieldToOne` and `MultiMint` extensions allow permissionlessly wrapping and unwrapping `PYUSDx`. On every `wrap()` + `unwrap()`, the principal decreases by 2 units, leading to a corresponding loss of (future) yield of 2.
2. If a ERC4626 vault, built around a `PYUSDx` balance, accounts for its `totalAssets` as `balanceOf(self) + accruedYieldOf(self)`, or equivalently `balanceWithYieldOf(self)`, the share price can be reduced with every deposit: for example, if the initial share price is 1, a depositor which adds 10^{-6} asset receives 10^{-6} share, but the `totalAssets` do not increase, since the 10^{-6} assets added are equaled by 10^{-6} of yield loss. Shares have been minted but `totalAssets` have not increased, so the share price has decreased: all shareholders (including the depositor) have incurred a loss. This is a griefing vector, and could be problematic in cases where the vault assumes that the share price is monotonically increasing.

Risk Accepted

M0 is aware of this issue and stated:

The rounding-down of principal on transfer is inherent to the indexed-balance design and the loss is bounded to 10^{-6} PYUSDx per transfer. The behavior will be documented so that integrators (e.g. ERC4626 vaults) account for it and follow the recommended integration patterns.

#012 Unsafe Casting

Correctness	Low	Version 1	Code Corrected 
-------------	-----	-----------	--

In `PYUSDx.sol`, the functions `_addEarningAmount()` and `_subtractEarningAmount()` update an account's `balance` and `earningPrincipal`. They rely on `_getPrincipalAmountRoundedDown()` and `_getPrincipalAmountRoundedUp()` to calculate the principal, which perform an unsafe casting of the input `amount`.

```
function _getPrincipalAmountRoundedDown(uint256 presentAmount, uint128 index) internal
    pure returns (uint112) {
    return IndexingMath.getPrincipalAmountRoundedDown(uint240(presentAmount), index);
}

function _getPrincipalAmountRoundedUp(uint256 presentAmount, uint128 index) internal
    pure returns (uint112) {
    return IndexingMath.getPrincipalAmountRoundedUp(uint240(presentAmount), index);
}
```

For `amount` exceeding 2^{240} , this truncation causes an incorrect principal amount to be calculated.

Code Corrected

`_getPrincipalAmountRoundedDown()` and `_getPrincipalAmountRoundedUp()` now use `UIntMath.safe240()` to prevent unsafe truncation.

#013 `__AccessControl_init()` Not Explicitly Called in Several Contracts

Informational	Version 1	Code Corrected 
---------------	-----------	--

`IssuerGateway`, `ExtensionFactory`, and `ExtensionBeacon` explicitly call `__AccessControl_init()` in their initializers, but `PYUSDx`, `Portal`, `SwapFacility`, `Extension`, and `BridgeAdapter` do not, despite all inheriting `AccessControlUpgradeable`. While the function is currently a no-op, the inconsistency is a maintenance risk: if a future OpenZeppelin version introduces meaningful initialization logic, the omission would silently produce uninitialized state.

Code Corrected

`__AccessControl_init()` is now explicitly invoked within the initialization logic of all contracts inheriting `AccessControlUpgradeable`.

#014 `lastIndexOf` And `lastUpdateTimestampOf` Not Updated on Self-Claim with Zero Fee

Informational

Version 1

Code Corrected ✓

`_claimFor()` updates `lastIndex` and `lastUpdateTimestamp` only as a side-effect of `_transfer()` calls. When an account is its own claim recipient and the fee is zero, both transfers are skipped, so `_updateIndexOf()` is never called. The two fields remain at their pre-claim values even though yield has been credited, making `lastIndexOf()` and `lastUpdateTimestampOf()` unreliable.

Code Corrected

`_claimFor()` now calls `_updateIndexOf()` unconditionally, ensuring `lastIndex` and `lastUpdateTimestamp` are always refreshed regardless of whether transfers are skipped.

#015 `RateLimiter` Remaining Amount Can Be Set Arbitrarily by Manager

Informational

Version 1

Specification Changed ✓

The `RATE_LIMITER_MANAGER_ROLE` can set an issuer's `remainingAmount` to arbitrary numbers with multiple calls to `setRateLimit()`.

For example:

Remaining amount can be decreased while not changing the capacity:

1. initial state: capacity = 10, remainingAmount = 10
2. setRateLimit with new capacity = 5: capacity = 5, remainingAmount = min(5, 10) = 5
3. setRateLimit with new capacity = 10: capacity = 10, remainingAmount = min(10, 5) = 5

Remaining amount can be increased unboundedly:

1. initial state: capacity = 10, remainingAmount = 10
2. setRateLimit with enabled = false : capacity = 0, remainingAmount = 0
3. setRateLimit with new capacity = 1000: capacity = 1000, remainingAmount = 1000

Specification Changed

The specification of the function behavior regarding remaining amount and capacity has been updated in `IRateLimiter.setRateLimit()` NatSpec.

#016 `setPayloadGasLimit()` Cannot Reset the Gas Limit to Zero

Informational

Version 1

Code Corrected ✓

`setPayloadGasLimit()` reverts on a zero value, so once a gas limit is set for a destination chain it can only be updated to another non-zero value. There is no way to unset or disable it to offboard a payload type or a chain.

Code Corrected

`setPayloadGasLimit()` now accepts a zero value, allowing the deactivation of gas limits for specific chains or payload types.

#017 Executed and Cancelled Proposals Are Deleted

Informational

Version 1

Acknowledged ☹

`IssuerGateway` deletes the `MintProposal` storage entry upon execution or cancellation. This makes it impossible to query historical proposals on-chain and complicates off-chain indexing, as observers must rely solely on emitted events to reconstruct the full proposal lifecycle.

Acknowledged

M0 is aware of this behavior and stated:

Deleting executed/cancelled `MintProposal` storage entries is intentional. The full proposal lifecycle is reconstructable from the emitted events, and off-chain UIs will be built to track and display mint proposals from those events.

#018 Inaccurate NatSpec

Informational

Version 1

Code Corrected ✓

Some functions have inaccurate NatSpec, below is a non-exhaustive list:

- `IExtensionFactory`: `extensionName` is documented as 'max 32 bytes' but nothing is enforcing it
- `ISwapFacility`: `recipient` in NatSpec of `replaceAsset*` (and internal `_replaceAsset()`) does not receive `amount` of `asset`, as `amount` is in `PYUSDx` and thus has 6 decimals. Likewise, in the NatSpec of internal `_swapInMultiMint()`, `recipient` does not receive `amount`, since that is denominated in the input token which has possibly different decimals.

Code Corrected

Inaccurate NatSpec entries have been corrected.

#019 Low-Level Call in `_wrap()` Can Be Forced into the Fallback Branch

Informational

Version 1

Acknowledged 

`Portal._wrap()` uses a low-level call with no explicit gas stipend to invoke `swapFacility.swapIn()`. Low-level calls forward at most 63/64 of the total gas, retaining 1/64 in the calling context. If a destination extension token consumes a large amount of gas, an attacker could craft a call that leaves just enough gas for the outer context to continue but not enough for `swapIn()` to complete. The low-level call would then return `false`, silently triggering the fallback branch and delivering plain `PYUSDx` to the recipient instead of the intended wrapped token.

Acknowledged

M0 is aware of this behavior.

#020 Missing Proposer Role Validation During Mint Execution

Informational

Version 1

Code Corrected 

In `IssuerGateway.sol`, the `mint()` function executes a pending proposal without verifying if the original proposer still retains the `OPERATOR_ROLE`.

If an operator is identified as malicious and has their `OPERATOR_ROLE` revoked, any proposals they created remain executable. Any executor could still process it later, effectively extending the malicious actor's power beyond their authorized window.

Code Corrected

The `mint()` function re-validates that the original proposer (`proposal.minter`) still holds `OPERATOR_ROLE` before execution.

#021 Restricted Cancellation Logic Leads to Dangling Proposals

Informational

Version 1

Code Corrected 

In `IssuerGateway.sol`, the `cancelMint()` function restricts cancellation rights exclusively to the original proposer. If an operator is removed from the protocol, all of their pending proposals become dangling. Because only the removed operator can call `cancelMint()`, there's no way to invalidate these potentially dangerous proposals. These dangling proposals remain in the system as executable risks.

Code Corrected

Resolved by re-validating the original proposer's `OPERATOR_ROLE` during `mint()`. Proposals from a revoked operator are no longer executable.

#022 Silent Mapping Updates in `setBridgeChainId()`

Informational

Version 1

Code Corrected 

Within the cross-chain architecture, bridge-related operations rely on a strict 1-1 correspondence between an `internalChainId` and a `bridgeChainId`. These routing mappings are managed by the `BridgeAdapter` contract. When a new pair is configured via `setBridgeChainId()`, it identifies and deletes the existing forward and reverse mappings before setting the new pair.

State Transition Example:

1. Initial State: Two valid 1-1 pairs exist.

- `$.internalToBridgeChainId[I1] = B1`, `$.bridgeToInternalChainId[B1] = I1`
- `$.internalToBridgeChainId[I2] = B2`, `$.bridgeToInternalChainId[B2] = I2`

2. Admin updates the connection to link `I1` to `B2` by calling `setBridgeChainId(I1, B2)`.

• Clean Up Old Forward/Reverse Mappings:

- `$.internalToBridgeChainId[I2] = 0`
- `$.bridgeToInternalChainId[B1] = 0`

• New Assignments

- `$.internalToBridgeChainId[I1] = B2`, `$.bridgeToInternalChainId[B2] = I1`

The current implementation only emits a single `BridgeChainIdSet(I1, B2)` event for the new connection. It fails to explicitly log the deletions of the old connections, thereby requiring off-chain indexers to actively track all relevant events and maintain connection states properly.

Code Corrected

`setBridgeChainId()` now explicitly emits events `BridgeChainIdRemoved` for the deleted forward and reverse mappings.

#023 Specification Mismatches

Informational

Version 1

Specification Changed 

The following are mismatches between the provided specification and the code under review:

1. The provided specification mentions: “`totalSupply` equals the sum of all account balances plus all unclaimed yield.”, but actually the unclaimed yield is not part of `totalSupply` in the code.
2. The specification mentions: “`lastIndex` only increases over time (for a positive rate)”. `lastIndex` actually reset to `1e12` once the user stops earning.
3. Proxies are indicated as `UUPS` in the specification, but they are actually instances of `TransparentUpgradeableProxy`.
4. The specification defines the invariant `sum(earningPrincipal) * currentIndex >= sum(balance)`, which does not align with the implementation. There is no global `currentIndex`, and the equation fails when no accounts are earning. Furthermore, even if the formula is adjusted to account for

individual indices and non-earning balances, the invariant would still break, as unrealized yield can be reduced during transfers, see [Transfers reduce unclaimed and future yield](#).

Specification Changed

The specifications have been updated and are now aligned with the implementation.

#024 Extension Cannot Be Used as Collateral Asset in MultiMint

Informational	Version 1	Acknowledged
---------------	-----------	--------------

MultiMint only rejects `address(0)` and `PYUSDx` itself as valid assets, so another extension token can be added as an asset. When a user then swaps that asset for the `MultiMint` extension token via `SwapFacility`, `_swap()` detects that both tokens are approved extensions and routes the call through `_swapExtensions()` instead of `_swapInMultiMint()`. This bypasses the `MultiMint._wrapAsset()` logic entirely.

Acknowledged

M0 is aware of this behavior and stated:

This is handled operationally: Asset Cap Managers should not add extensions as collateral assets in MultiMint. Enforced as an operational guideline for the Asset Cap Manager role.

#025 Frozen PYUSDx Account Can Still Swap Between Extensions

Informational	Version 1	Acknowledged
---------------	-----------	--------------

`_swapExtensions()` unwraps the input extension into `PYUSDx` and immediately re-wraps it into the output extension, transferring `PYUSDx` through the `SwapFacility` in the process. Since the frozen check only exists in `PYUSDx._transfer()` and not in `SwapFacility`, an account frozen on `PYUSDx` can still perform extension-to-extension swaps.

Acknowledged

M0 is aware of this behavior and stated:

Ideally, a frozen account should be frozen at the extension level in addition to the `PYUSDx` level. The Freeze Manager is expected to apply freezes consistently across `PYUSDx` and the relevant extensions to fully contain a frozen account.

#026 Missing Consistency Check for Beacons in ExtensionFactory

Informational	Version 2	Code Corrected
---------------	-----------	----------------

The `ExtensionFactory` constructor only verifies that the provided `yieldToOneBeacon_` and `multiMintBeacon_` were not the zero address. It fails to validate if their internal `pyusdx` and

`swapFacility` references matched the factory's configuration. A misconfigured beacon would propagate mismatched dependencies to all deployed proxies.

Code Corrected

The `_revertIfInvalidBeacon()` helper was added to ensure all beacons are correctly wired to the matching `pyusdx` and `swapFacility` instances.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.